

Haskell Engine

Architecture and Reference

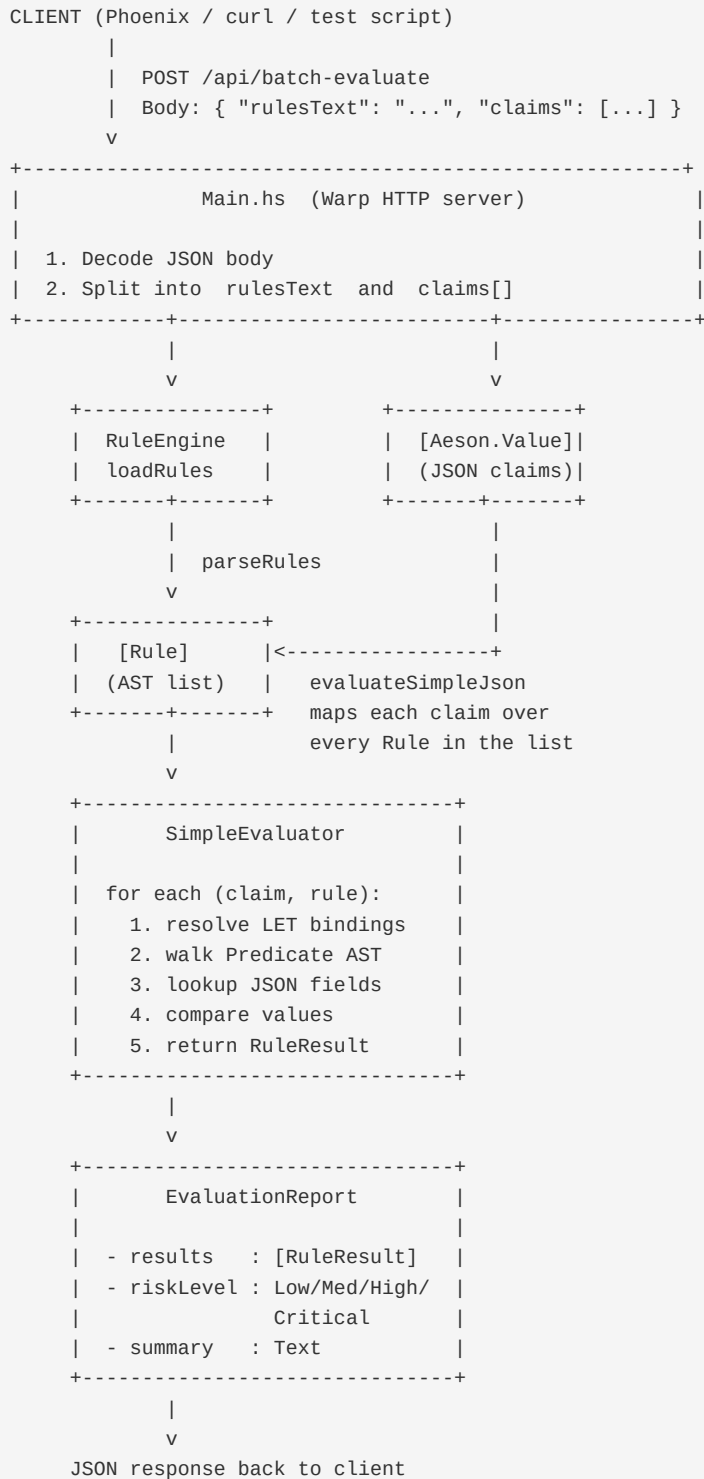
Functional Programming Design

The rule evaluation engine is written in Haskell, a purely functional language.

- Rules are parsed into an AST and evaluated via pattern matching and recursive descent.
- `src/Claims/Syntax.hs` defines the DSL grammar as algebraic data types (ADTs) -- a hallmark FP design.
- `src/Claims/RuleEngine.hs` and `src/Claims/SimpleEvaluator.hs` compose rule evaluation functionally, with no mutable state.
- `src/Claims/PolicyCombiner.hs` combines rule results using combinators -- a classic FP pattern.

The Elixir/Phoenix frontend is also functional: Phoenix LiveView uses immutable state diffs and message-passing, and the claims context uses pipelines (`|>`) and pure transformations. This split is deliberate -- the Haskell engine handles rule logic (pure, stateless, easily testable) while Elixir handles web and state management. Side effects (DB, HTTP) are pushed to the edges; the rule evaluation core remains pure.

Big Picture



Evaluation Pipeline

Step 1 -- HTTP Request and Routing

File: app/Main.hs

The Warp web server receives the POST. pathInfo splits the URL into a list; Haskell's case expression matches it to the right handler -- no framework, just pattern matching.

```

case pathInfo request of
  ["api", "batch-evaluate"] -> handleBatchEvaluate request respond
  ["api", "evaluate"]       -> handleEvaluate      request respond
  ["api", "compile-rules"]  -> handleCompileRules cache request respond
  ["api", "parse-rule"]     -> handleParseRule     request respond
  ["api", "health"]         -> handleHealth        respond
  _                         -> respond $ responseLBS status400 [] "Not found"

```

The handler decodes the raw HTTP body into a `BatchEvaluationRequest`:

```

data BatchEvaluationRequest = BatchEvaluationRequest
  { batchRulesText :: Text          -- the DSL rule program
  , batchClaims    :: [Aeson.Value] -- list of JSON claim objects
  }

```

Step 2 -- Parsing DSL Text into an AST

File: `src/Claims/RuleEngine.hs`

```

loadRules :: Text -> Either String RuleEngine
loadRules rulesText = case parseRules rulesText of
  Left err  -> Left ("Parse error: " ++ show err)
  Right rules -> Right (RuleEngine rules)

```

`parseRules` reads raw DSL text and produces `[Rule]` -- in-memory Haskell data structures. `Either` is Haskell's "success or failure" type: `Left err` on failure, `Right rules` on success. After this step the text is gone; the engine works entirely with the AST.

Step 3 -- Rule (AST) Structure

File: `src/Claims/Syntax.hs`

A Rule is a Haskell record (like a struct) with five fields:

```

Rule
+-- ruleName      : Text          e.g. "high_amount"
+-- ruleDescription : Text        e.g. "Flag claims over $10,000"
+-- ruleBindings  : [Binding]    optional LET x = some.field aliases
+-- ruleCondition  : Predicate    <-- the boolean expression tree
+-- ruleAction     : Action       what to do when condition is true

```

WHEN amount > 10000 THEN FLAG_FRAUD "large" becomes:

```

ruleCondition = GreaterThan
  (Field "amount")    <-- field reference
  (NumberValue 10000.0) <-- literal number

ruleAction     = FlagFraud "large"

```

Predicate is a tree -- complex rules nest predicates inside each other:

```
WHEN amount > 10000 AND status = "pending"
```

```

      And
    /   \
GreaterThan Equals
(amount, (status,
10000)   "pending")

```

Step 4 -- Evaluating Each Claim

File: `src/Claims/RuleEngine.hs`

```

evaluateSimpleJson :: RuleEngine -> Aeson.Value -> EvaluationReport
evaluateSimpleJson engine claimDoc =
  let results = map (evaluateRuleSimple claimDoc) (engineRules engine)

```

`map f list` applies `f` to every element. Every rule is always evaluated -- there is no short-circuit on first match. All results are collected and `determineRiskLevel` picks the worst outcome.

```

rules = [ rule1, rule2, rule3, ... ]
          |       |       |
          v       v       v
          evaluateRuleSimple claim
          |       |       |
          v       v       v
results = [ result1, result2, result3, ... ]

```

Step 5 -- Walking the Predicate Tree

File: `src/Claims/SimpleEvaluator.hs`

The evaluator uses an `EvalEnv` (evaluation environment) that carries three pieces of context:

```

EvalEnv
+-- envLetBindings : Map Text Text           -- LET aliases
+-- envScopeVars   : Map Text Aeson.Value    -- named quantifier variables
+-- envDoc          : Aeson.Value            -- current JSON document context

```

Field resolution checks scope variables first (for named quantifier bindings), then LET bindings, then the document. This enables nested quantifiers to access both inner and outer bound elements -- the key mechanism for first-order predicate calculus equivalence.

The core `evaluateWithEnv` function is recursive, pattern-matching on each predicate node:

Predicate node	What the evaluator does
GreaterThan field val	look up field in JSON -> parse as number -> compare
Equals field val	look up field in JSON -> compare as string
And p1 p2	evaluate p1 AND evaluate p2 (recurse both)
Or p1 p2	evaluate p1 OR evaluate p2 (recurse both)
Not p	NOT (evaluate p) (recurse one branch)
Between field lo hi	look up field -> check lo <= field <= hi
HasDiagnosis "code"	search <code>diagnosis_codes[*].code</code> in claim

Predicate node	What the evaluator does
HasProcedure "code"	search service_lines[*].procedure_code
Exists (Just x) path p	bind each element to x -> evaluate p
ForAll (Just x) path p	bind each element -> check ALL satisfy p
IsNull field	field missing or null -> true
HelperCall "is_weekend"	extract date field -> check day-of-week

Step 6 -- JSON Field Lookup

File: src/Claims/SimpleEvaluator.hs

Field references are resolved by navigating the JSON object tree using dot-separated path segments. Array elements are accessed by numeric index.

DSL field reference	Path walked	Value returned
Field "amount"	top level	"15000"
Field "claim.amount"	claim -> amount	"9"
SegmentField "CLM" "01"	CLM -> 01	"X"
Field "items.0.price"	array index 0 -> field	"5"

All JSON leaf values are coerced to Text (plain string) for comparison:

JSON type	Becomes
"pending"	"pending"
15000	"15000.0"
true	"true"
null	Nothing (field treated as missing)

Step 7 -- Result and Risk Aggregation

File: src/Claims/RuleEngine.hs

Each rule produces a RuleResult:

```
RuleResult
  +- resultRuleName : Text      "high_amount"
  +- resultMatched   : Bool     True / False
  +- resultAction    : Maybe Action Just (FlagFraud' "large") or Nothing
  +- resultDetails   : Text     "Rule matched: ..."
```

determineRiskLevel scans all results and picks the worst:

```
Any FlagFraud or RejectClaim action?  -> CriticalRisk
3 or more rules scored >= 70?         -> HighRisk
1 or 2 rules scored >= 70?            -> MediumRisk
Nothing significant triggered?         -> LowRisk
ApproveClaim only?                   -> LowRisk (does not elevate risk)
```

The final EvaluationReport is serialised to JSON and returned in the HTTP response.

Rule Storage

There are two distinct places rules can live, depending on which API endpoint is used.

Storage 1 -- RuleEngine list (per-request, temporary)

File: `src/Claims/RuleEngine.hs`

```
data RuleEngine = RuleEngine
  { engineRules :: [Rule]  -- a plain Haskell list of Rule ASTs
  }
```

Used by `/api/batch-evaluate`. The DSL text is parsed fresh on every request and stored here for the lifetime of that one request. When the HTTP response is sent the list is discarded -- nothing persists.

Storage 2 -- CompiledRuleCache (server-lifetime, persistent)

File: `src/Claims/RuleCache.hs`

```
data CompiledRuleCache = CompiledRuleCache
  { cacheRules :: TVar (Map Text CompiledRule)
  }
```

This map lives for the entire server lifetime, allocated once in main and shared across all requests:

```
main :: IO ()
main = do
  cache <- newCompiledRuleCache  -- created once
  run 8080 (app cache)           -- shared across all requests
```

Rules enter the cache via `/api/compile-rules`. Each entry is a `CompiledRule`:

```
data CompiledRule = CompiledRule
  { compiledRuleName :: Text
  , compiledFunction :: Aeson.Value -> RuleResult  -- claim in, result out
  , compiledAt       :: UTCTime
  }
```

`compiledFunction` is a pre-built evaluation function -- the AST is already bound inside it, so there is no re-parsing on each claim. `TVar` is Haskell's transactional variable; reads and writes are wrapped in atomically, making the cache thread-safe.

Storage Comparison

	RuleEngine list	CompiledRuleCache
File	<code>RuleEngine.hs</code>	<code>RuleCache.hs</code>
Lives for	One request	Entire server lifetime
Populated by	<code>/api/batch-evaluate</code>	<code>/api/compile-rules</code>
Storage type	<code>[Rule]</code> -- plain list	<code>TVar (Map Text CompiledRule)</code>
Thread safe?	N/A (request-local)	Yes -- atomically via STM
Re-parses DSL?	Every request	No -- parsed once

Claim Structure

What a decoded medical claim looks like. Examples from test/fixtures/claims/.

Example 1 -- Normal / Approved Claim

Source: claim_normal_approved.json -- A low-value office visit (\$150) with a valid authorisation and a known provider.
Expected result: APPROVED.

```
{
  "claim_id": "CLM-NORMAL-001",
  "provider": {
    "name": "Kansas City Medical Center",
    "npi": "1234567901", "type": "Clinic", "state": "MO",
    "risk_score": 25, "tenure_days": 800
  },
  "patient": {
    "name": { "first": "Elizabeth", "last": "Wilson" },
    "date_of_birth": "1982-10-12", "gender": "Female"
  },
  "claim_details": { "admission_date": "2026-01-27",
    "discharge_date": "2026-01-27", "service_type": "Office Visit" },
  "diagnosis_codes": [
    { "code": "Z00.00", "description": "General adult exam" }
  ],
  "service_lines": [
    { "procedure_code": "99213", "units": 1, "line_amount": 150.00 }
  ],
  "financial": { "claim_amount": 150.00, "patient_copay": 25.00 }
}
```

Example 2 -- High-Amount Fraud Trigger

Source: claim_high_amount_trigger.json -- A \$75,000 hospital inpatient claim. Expected result: FLAG_FRAUD.

```
{
  "claim_id": "CLM-HIGH-AMOUNT-001",
  "claim": { "amount": 75000, "units": 10, "drg_code": "640" },
  "provider": {
    "name": "Premium Medical Center", "npi": "1234567890",
    "type": "Hospital", "state": "CA", "risk_score": 25
  },
  "patient": {
    "name": { "first": "John", "last": "Smith" },
    "date_of_birth": "1981-03-15", "gender": "Male"
  },
  "diagnosis_codes": [
    { "code": "E11.9", "description": "Type 2 diabetes" },
    { "code": "I10", "description": "Essential hypertension" }
  ],
  "service_lines": [
    { "procedure_code": "99213", "units": 5, "line_amount": 750.00 },
    { "procedure_code": "80053", "units": 1, "line_amount": 85.00 }
  ],
  "financial": { "claim_amount": 75000.00 }
}
```

Aeson Internal Representation

Aeson is the standard Haskell library for JSON parsing and encoding. It defines a Value type that mirrors the JSON data model exactly, so every JSON document can be represented as a Haskell value without information loss.

JSON type	Aeson constructor	Example
{ "key": ... }	Aeson.Object (KeyMap)	the whole claim document
[...]	Aeson.Array (Vector)	diagnosis_codes, service_lines
"some text"	Aeson.String Text	"CLM-HIGH-AMOUNT-001"
75000	Aeson.Number Scientific	75000
true / false	Aeson.Bool Bool	true
null	Aeson.Null	null

Field Extraction by DSL Reference

lookupJsonField splits a dot-path into segments and walks the tree:

DSL field reference	Path walked	Value returned
Field "claim_id"	top level	"CLM-HIGH-AMOUNT-001"
Field "claim.amount"	claim -> amount	"75000.0"
Field "provider.state"	provider -> state	"CA"
Field "service_lines.0.procedure_code"	array idx 0 -> field	"99213"
Field "service_lines.1.units"	array idx 1 -> field	"1.0"

End-to-End Example

Input (POST body)

```
{
  "rulesText": "RULE high_amount \"High value claim\" WHEN amount > 10000
    THEN FLAG_FRAUD \"Unusually large amount\";",
  "claims": [
    { "amount": 15000, "provider": "NPI123", "status": "pending" }
  ]
}
```

Pipeline Trace

```

rulesText --> parseRules --> Rule { ruleName      = "high_amount"
                                     , ruleCondition = GreaterThan (Field "amount")
                                           (NumberValue 10000)
                                     , ruleAction    = FlagFraud "Unusually large amount"
                                     }

claim JSON --> evaluateRuleSimple
    |
    +- lookupJsonField (Field "amount") claim
    |   +-- navigates { "amount": 15000 } -> returns "15000.0"
    |
    +- textToDouble "15000.0" -> 15000.0
    |
    +- 15000.0 > 10000.0 = TRUE
    |
    +- RuleResult { resultRuleName = "high_amount"
                   , resultMatched  = True
                   , resultAction   = Just (FlagFraud' ...)
                   , resultDetails  = "Rule matched: High value claim"
                   }

determineRiskLevel [matched result with FlagFraud] -> CriticalRisk

```

Output (response body)

```

{
  "batchResults": [
    {
      "claimIndex": 0,
      "report": {
        "results": [
          {
            "resultRuleName": "high_amount",
            "resultMatched": true,
            "resultAction": { "tag": "FlagFraud'",
                           "contents": "Unusually large amount" },
            "resultDetails": "Rule matched: High value claim"
          }
        ],
        "totalRules": 1,
        "matchedRules": 1,
        "overallRisk": "CriticalRisk",
        "summary": "Evaluated 1 rules, 1 matched. Overall risk: CriticalRisk"
      }
    ],
    "totalClaims": 1
  }
}

```

Key Haskell Concepts

Concept	What it does	Where you see it
case ... of	pattern match -- like switch but exhaustive	routing, predicate walking
Either L R	success (Right) or failure (Left)	loadRules, parseRules
Maybe a	value might be absent (Just x or Nothing)	field lookup results

Concept	What it does	Where you see it
map f list	applies f to every item in a list	fanning claims over rules
mapM f list	like map but allows IO side effects	compileRules
[Rule]	a list of Rules	the engine's rule store
TVar	thread-safe transactional variable	CompiledRuleCache
Recursive ADT	a type that references itself	Predicate tree (And, Or, Not)
Record syntax	named fields in a data type (struct)	Rule, RuleResult